OAuth 2.0
Application
Sign UP
username
password
email
mobile
one way
Sign in with Google
Sign in with Facebook
2nd way
Google
3rd party app
access token
email
mobile
consider this data is required by 3rd party and access token is passed by google which has this information
New topic
Auto Complete Engine
other name :
Design top k most searched Queries.
Google
O(1)
sw
swing -> 1000
swagger -> 800
swan -> 700
swim
Q : How many autocomplete suggestion is needed ?
A : 6 auto complete suggestions
Q: How to decide the search result ?
A: based on popularity (frequency)
Q: Search key length ?
A : 100
Q: Which languages are supported ?
A: English
Q: Should we need to sort?
A : yes
Q: Case sensitive:
A : no
Q: Daily Active Users ?
A : 5 million DAU
Auto Complete
Data Gathering Service
Data Query Service
Date Gathering Service :
Responsible for collecting information.
Data Query Service:
Given a search Query or prefix, return top 6 most searched result.
Data gathering service:
SQI DB.
Basic Frequency table: (Initially Empty)
Query frequency
switch 2
swing 4
swim 3
swirl 3
swan 6
Data Query Service :
search : sw
swim
swan
swing
swirl
switch
How OS works?
select * from frequency
where query like 'prefix%' {sw}
order by frequency DESC
limit 6;
Work is Done.
Dataset is small -> then this sol is OK [O(n)]
Data is huge -> Very very slow.
Next Solution :
Tries Data Structure.
English : 26
sw?
swim
swing
swan
swirl
s
a
i
w
m (7)
n
n (6)
r
g (4)
t (3)
Tries
{
Tries * Children[26];
bool eow; end of word
int freq;
}
Problems with Normal Tries :
faster data structure to search strings
Time complexity :
O(p) + O(c) + O(c log c)
p : length of the prefix to be searched.
n : Total no. of nodes in tries.
c : No. of children of a given node
high time complexity comparatively
we are looking for O(1)
Top K most frequent queries :
1. Find the prefix. -> O(p)
2. Traverse the subtree from this node to get
all the valid children -> O(c)
3. Sort the result on the basis of freq. -> O(c Log C)
worst case -> traverse complete tries
How to make this fast ?
1. Limit the max length of query (100 length max)
2. Cache top 6 results for each node.
Limit the max length of string :
Users rarely search for very long string :
p -> is small (say 100 chars at max)
O(p) -> O(1)
Cache top 6 results
k is small -> which is 6.
Cache them at each node.
sw?
swim
swing
swan
swirl
s
cache
swim(7),
swan(6),
swing(4),
swirl(3)
a
i
w
m (7)
n
n (6)
r
g (4)
t (3)
Top K most frequent queries :
1. Find the prefix. -> O(1)
3. Sort the result on the basis of freq. -> O(1)
Already sorted.
since data is stored in cache this issue is fix
Total complexity -> O(1) + O(1) = O(1)
Trade off -> space increase to dec time complexity.
How to Cache ??
Data Gathering Service:
Real time vs almost real time system
updating this for every frequency updating this for (one to ten bunks)
Real time Search prediction -> Almost impossible because throughput increases
Almost real time system:
Why ?
Top suggestions may not change much. like google search bar
Not practical to update out tries at every request.
Real time vs Timely updates.
twitter -> may use a more real time system then our engine.
But not for us (Google search)
Analytics logs
Raw table about search Queries.
Not indexed.
SQI Table : append only.
Query timestamp
swing. Aug 11 2025 22:48:03
swim. Aug 11 2025 22:48:05
swing. Aug 12 2025 22:48:34
aggregate the above data
Analytics logs
Aggregator Service
Aggregate d Data
Aggregator Service:
Aggregate data to be processed further (format correction)
non real time system -> aggregate once a week.
for more real time system -> aggregate once a day.
Aggregated data: SQI Table
Query. time. freq
swing. 2
swim. 4
updated time of every timestamp
Analytics logs
Aggregator Service
Aggregate d Data
Trie Cache
Trie DB
Workers
this cache is different that the cache that is at every node
Workers:
set of servers that perfrom asyne operations
to build our Tire OS and store it in Tire Db.
trie Cache:
Keep tries in memory for faster read.
Hold the weekly updated information.
Tries DB: (NoSQL)
persistent storage.
Tries -> key, value.
(MongoDb/ Casandra)
Mongo / casandra
key value
sw swim, swan, swing, swirl
swi swing, swirl, swim
swim
sw?
swim
swing
swan
swirl
s
cache
swim(7),
swan(6),
swing(4),
swirl(3)
a
i
w
m (7)
n
n (6)
r
g (4)
t (3)
Trie -> serialize -> key, value pair. (Hashmap)
key : Every prefix in the tries will be my key.
value : Data for each trie node (top 6 results)
First time service run : initially DB is empty
1 lakh analytics log
in one week.
10K aggregated data
Build tries
out of it..
trie DB
trie Cache
Client
LB
Servers
/search/query=sw
trie Cache
Trie DB
FLOW :
1. A search query is sent to the LB.
2. The LB route this to out server.
3. server tries to fetch data from cache DB.
If exist
4. If not exist, replenish data from DB to cache.
(If cache is full, or cache is down)
Further Optimizations??
1. client
/search/query=sw (AJAX)
/search/query=sw (AJAX)
/search/query=swin (AJAX)
HTTP Req (not one)
AJAX Req (use one)
when request are sent repeatedly
Browser -> sent ajax req, for auto complete.
It will not refresh full web page at each letter.
2. To decrease the load on our server:
We can also cache results at broser side (client side)
3. offensive/ abusive words can also come.
API Servers
Kafka
Trie Cache
Set of rules.
4. SCaling th DB
Sharding key ??
- Shard on the basis of first english character
Sharding??
DB1
DB26
words with
But, x -> less
words with a -> more
"a" server has more load, but we need uniform load on all server
Shard Mapping Table
a
as to am
an to az
splitting this server would not uniform load on all servers
web servers
DB
single server
5. Multiple language ??
Hinglish -> hindi in english letters
Unicodes charcters -> encoding standards
for all characters of every language in the world.
6. Diff country -> diff servers.
We can have country specific top results.
CDN (Cache) for better response.